

# Preliminares

Noel Pérez Calvo

28 de mayo de 2026

## 1. Preliminares

Este capítulo establece los fundamentos teóricos necesarios para abordar el problema de descubrir expresiones analíticas de los ceros de sistemas de ecuaciones no lineales mediante regresión simbólica. En la primera parte (1.1) se introduce la notación y las propiedades de los sistemas de ecuaciones no lineales paramétricos, destacando la multiplicidad de soluciones que pueden existir para una misma tupla de parámetros; se describen los métodos numéricos para el cálculo de ceros de tales sistemas, tanto los métodos iterativos clásicos como el método híbrido de Powell implementado en la biblioteca `SciPy` [15], y se definen formalmente los conceptos de distancia euclidiana y producto cartesiano, que serán utilizados en la generación del conjunto de datos de entrenamiento. Posteriormente, en la sección 1.2 se abordan los principios de la regresión simbólica [9], una técnica de aprendizaje automático que permite inferir expresiones algebraicas a partir de datos, y se explican los fundamentos de la programación genética, las funciones de pérdida que guían el proceso evolutivo y los parámetros que controlan el comportamiento del algoritmo. Por último, se presentan los fundamentos de la prueba t de Student para muestras pareadas, que se empleará en el capítulo de experimentación para comparar estadísticamente las distintas configuraciones del algoritmo.

### 1.1. Sistemas de ecuaciones no lineales paramétricos

En esta sección se formaliza el concepto de sistema de ecuaciones no lineales dependiente de parámetros, así como las herramientas numéricas y matemáticas necesarias para su estudio.

Un sistema de ecuaciones no lineales paramétrico se expresa como la búsqueda de vectores  $\mathbf{x} \in \mathbb{R}^n$  que satisfacen

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}, \tag{1}$$

donde:

- $\mathbf{F} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^n$  es una función vectorial no lineal,
- $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$  es el vector de variables incógnita,
- $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^T \in \mathbb{R}^p$  es el vector de parámetros del modelo,
- $\mathbf{0} \in \mathbb{R}^n$  denota el vector nulo.

La notación  $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})$  indica que  $\mathbf{F}$  depende explícitamente de las variables  $\mathbf{x}$  y de los parámetros  $\boldsymbol{\theta}$ , estos últimos se consideran fijos durante el proceso de resolución del sistema de ecuaciones.

**Ejemplo 1.** Considérese el sistema de dos ecuaciones

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \begin{pmatrix} (x_1 - a)(x_2 - ab) \\ x_1x_2 - ab^2 \end{pmatrix} = \mathbf{0},$$

con  $\mathbf{x} = (x_1, x_2)^T \in \mathbb{R}^2$  y parámetros  $\boldsymbol{\theta} = (a, b)^T$ ,  $a > 0$ ,  $b > 0$ .

Este sistema posee dos soluciones reales para cada tupla de parámetros:  $\mathbf{x} = (a, b^2)$  y  $\mathbf{x} = (b, ab)$ . Nótese que la primera componente de la solución  $(a, b^2)$  anula el factor  $(x_1 - a)$  de la primera ecuación, mientras que en la solución  $(b, ab)$  es la segunda componente la que anula el factor  $(x_2 - ab)$ . Esta multiplicidad de ceros y su dependencia explícita de los parámetros ilustran el tipo de fenómenos que aparecen en sistemas no lineales paramétricos y motivan la necesidad de determinar, para cada valor de los parámetros, todas las raíces que serán utilizadas en la construcción del conjunto de datos de entrenamiento con el que se alimentará la regresión simbólica.

La no linealidad de  $\mathbf{F}$  implica que, a diferencia de los sistemas lineales, no existe un método directo general para obtener todas las soluciones, y la estructura del conjunto solución puede ser compleja. Por esta razón se recurre a métodos numéricos iterativos, que se describen en la siguiente sección.

### 1.1.1. Métodos numéricos para el cálculo de ceros

Una vez definido el sistema no lineal paramétrico, es necesario obtener numéricamente sus soluciones para distintas tuplas de parámetros. Dado que, salvo en casos triviales, no es posible despejar analíticamente las raíces de un sistema no lineal arbitrario, se recurre a métodos iterativos que aproximan las soluciones de manera sucesiva. A continuación se describen dos de los métodos más utilizados, que serán empleados en la etapa de generación de datos, es decir, en la construcción del conjunto de entrenamiento que alimentará la regresión simbólica.

Para sistemas no lineales generales, la obtención de soluciones explícitas mediante métodos simbólicos no es factible, ya que la mayoría de estos sistemas carecen de soluciones expresables en forma cerrada [3]. En su lugar, se emplean métodos numéricos iterativos que, partiendo de una aproximación inicial  $\mathbf{x}^{(0)}$ ,

generan una sucesión  $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$  que converge a una solución  $\mathbf{x}^*$ , siempre que la aproximación inicial esté suficientemente cerca de dicha solución y que  $\mathbf{F}$  sea diferenciable con jacobiano no singular en  $\mathbf{x}^*$  [11].

El método de Newton-Raphson para sistemas no lineales [3] se define mediante la iteración

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - [\mathbf{J}_{\mathbf{F}}(\mathbf{x}^{(k)})]^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}), \quad (2)$$

donde:

- $\mathbf{x}^{(k)}$  es la aproximación en la iteración  $k$ ,
- $\mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta})$  es el valor de  $\mathbf{F}$  evaluado en dicha aproximación,
- $\mathbf{J}_{\mathbf{F}}(\mathbf{x})$  es la matriz jacobiana de  $\mathbf{F}$  respecto de  $\mathbf{x}$ , de dimensión  $n \times n$ , con entradas  $(\mathbf{J}_{\mathbf{F}})_{ij} = \frac{\partial F_i}{\partial x_j}$ .

Si  $\mathbf{F}$  es diferenciable y  $\mathbf{J}_{\mathbf{F}}$  es no singular en una solución  $\mathbf{x}^*$ , entonces existe un entorno de  $\mathbf{x}^*$  donde el método converge cuadráticamente, es decir,  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq C \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$  para alguna constante  $C > 0$  [11].

Los métodos cuasi-Newton [11] reducen el costo computacional al evitar el cálculo exacto del jacobiano en cada iteración. En lugar de ello, construyen una aproximación  $\mathbf{B}_k$  de la matriz jacobiana que se actualiza con información de los gradientes evaluados en iteraciones previas. Las actualizaciones típicas son de rango uno o dos, lo que significa que modifican la matriz anterior sumando una matriz de rango uno o dos, respectivamente. La iteración toma la forma

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{B}_k^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}),$$

donde  $\mathbf{B}_k$  se actualiza mediante fórmulas como la de Broyden [2].

La actualización de Broyden está definida por

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{(\mathbf{y}_k - \mathbf{B}_k \mathbf{s}_k) \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{s}_k},$$

donde  $\mathbf{s}_k = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$  y  $\mathbf{y}_k = \mathbf{F}(\mathbf{x}^{(k+1)}) - \mathbf{F}(\mathbf{x}^{(k)})$ .

Estos métodos alcanzan una convergencia superlineal [11]: la tasa de reducción del error es más rápida que la lineal pero sin llegar a la velocidad cuadrática del método de Newton.

En este trabajo se utilizará un método híbrido, como el implementado en la biblioteca SciPy [15], que combina una estrategia cuasi-Newton con búsqueda de paso para mejorar la robustez.

La búsqueda de paso consiste en ajustar la longitud de la corrección que se aplica en cada iteración para garantizar una disminución suficiente de la norma

del residuo  $\|\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})\|_2$ , que actúa como función de mérito. Formalmente, en cada iteración se determina un parámetro  $\lambda_k > 0$  tal que

$$\|\mathbf{F}(\mathbf{x}^{(k)} + \lambda_k \mathbf{p}_k; \boldsymbol{\theta})\|_2 < \|\mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta})\|_2,$$

donde  $\mathbf{p}_k = -\mathbf{B}_k^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta})$  es la dirección de búsqueda. Este procedimiento mejora la convergencia global del método.

A partir del método numérico descrito se generará el conjunto de datos de entrenamiento para la regresión simbólica, el cual está formado por múltiples tuplas  $(\boldsymbol{\theta}, \mathbf{x}^*)$ , donde  $\mathbf{x}^*$  es una raíz obtenida numéricamente para el parámetro  $\boldsymbol{\theta}$ .

Cuando se ejecuta el método desde varios puntos iniciales, es posible que dos de ellos converjan a la misma solución real, por lo que esta aparecerá repetida en la lista de soluciones encontradas. Para eliminar estas repeticiones se utiliza la distancia euclidiana, que se estudia en la siguiente sección.

### 1.1.2. Distancia euclidiana

En la etapa de resolución numérica, dos raíces obtenidas a partir de puntos iniciales distintos pueden corresponder al mismo cero del sistema, diferenciándose únicamente por errores de redondeo o por las tolerancias de convergencia de los métodos iterativos. Para decidir si dos soluciones deben considerarse iguales, se utiliza la distancia euclidiana.

**Definición 1** (Distancia euclidiana). Sean  $\mathbf{u} = (u_1, \dots, u_n)^T \in \mathbb{R}^n$  y  $\mathbf{v} = (v_1, \dots, v_n)^T \in \mathbb{R}^n$  dos vectores. La **distancia euclidiana** entre  $\mathbf{u}$  y  $\mathbf{v}$  se define como la norma euclidiana de su diferencia:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}. \quad (3)$$

La distancia euclidiana constituye una métrica sobre  $\mathbb{R}^n$ . En este trabajo, dos soluciones  $\mathbf{x}$  y  $\mathbf{x}'$  se consideran numéricamente equivalentes si su distancia euclidiana es menor que una tolerancia prefijada  $\tau > 0$ , lo que típicamente ocurre cuando ambas corresponden al mismo cero del sistema pero difieren debido a la acumulación de errores numéricos.

En particular, la **norma euclidiana** de un vector  $\mathbf{v} \in \mathbb{R}^n$  se define como su distancia al vector nulo, es decir,

$$\|\mathbf{v}\|_2 = d(\mathbf{v}, \mathbf{0}) = \sqrt{\sum_{i=1}^n v_i^2}. \quad (4)$$

Esta cantidad resulta útil para medir qué tan cerca está un punto de ser solución del sistema: si  $\|\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})\|_2$  es pequeño, entonces  $\mathbf{x}$  está próximo a anular todas las ecuaciones.

Una vez identificadas las soluciones distintas para cada tupla de parámetros, es necesario organizar sistemáticamente los valores de dichos parámetros sobre los que se evaluará el sistema. Esta organización se realiza mediante el producto cartesiano, que se define a continuación.

### 1.1.3. Producto cartesiano

Para construir el conjunto de datos de entrenamiento se debe evaluar el sistema no lineal en una colección representativa de valores de los parámetros. Con este fin se selecciona, para cada parámetro  $\theta_i$ , un conjunto finito de valores dentro de su rango admisible; el producto cartesiano de estos conjuntos proporciona todas las combinaciones posibles de parámetros que se utilizarán en la etapa de resolución numérica.

**Definición 2** (Producto cartesiano). *Sean  $A_1, A_2, \dots, A_m$  conjuntos arbitrarios. El **producto cartesiano** de estos conjuntos se define como el conjunto de todas las  $m$ -tuplas ordenadas  $(a_1, a_2, \dots, a_m)$  tales que  $a_i \in A_i$  para cada  $i = 1, \dots, m$ . Formalmente:*

$$A_1 \times A_2 \times \dots \times A_m = \{(a_1, a_2, \dots, a_m) \mid a_i \in A_i, i = 1, \dots, m\}. \quad (5)$$

Si cada  $A_i$  es un conjunto finito, el cardinal del producto cartesiano es el producto de los cardinales:  $|A_1 \times \dots \times A_m| = \prod_{i=1}^m |A_i|$ . En el contexto de este trabajo, cada  $A_i$  es un conjunto de valores del parámetro  $\theta_i$ , típicamente elegidos de manera equiespaciada dentro de un intervalo  $[\theta_i^{\min}, \theta_i^{\max}]$ . El producto cartesiano de estos conjuntos genera todas las tuplas  $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)$  que serán utilizadas como entrada del sistema durante la generación de datos, garantizando una cobertura uniforme de la región de interés en el espacio de parámetros.

**Ejemplo 2 (proceso de generación de datos).** Retomando el sistema del Ejemplo 1, con parámetros  $a$  y  $b$ , se requiere que cada uno tome valores en el intervalo  $[0.1, 5]$ .

Para obtener un conjunto finito de combinaciones de parámetros que cubran adecuadamente ese intervalo, se discretiza el rango de cada parámetro mediante 50 puntos equiespaciados, obteniéndose dos conjuntos finitos  $\Theta_a$  y  $\Theta_b$ . El producto cartesiano  $\Theta_a \times \Theta_b$  contiene 2 500 tuplas distintas. Para cada una de estas tuplas se ejecutará el método numérico desde varios puntos iniciales aleatorios, generando así el conjunto de pares  $(\boldsymbol{\theta}, \mathbf{x})$  que forman el conjunto de entrada de la regresión simbólica. Este mismo procedimiento se generaliza de manera directa a cualquier número de parámetros.

Una vez definidas las herramientas para generar los valores de los parámetros y para identificar las raíces del sistema, se dispone del marco necesario para producir el conjunto de datos con el que se alimentará el algoritmo de regresión simbólica, cuyos fundamentos se exponen en la siguiente sección.

## 1.2. Regresión simbólica

La regresión simbólica es la técnica central sobre la que se apoya la metodología de este trabajo. A continuación se presentan sus fundamentos teóricos y los elementos que la componen.

### 1.2.1. Concepto y objetivos

La regresión simbólica es una técnica de aprendizaje automático que descubre, a partir de un conjunto de datos observados, una expresión matemática cerrada. Dicha expresión describe la relación funcional entre las variables de entrada y la variable de salida [9]. A diferencia de la regresión tradicional, donde la estructura del modelo se fija de antemano y solo se ajustan los parámetros, la regresión simbólica explora simultáneamente el espacio de formas funcionales y de parámetros numéricos [12].

Existen diversas implementaciones de regresión simbólica, muchas de las cuales se basan en programación genética, enfoque que se describe en la siguiente subsección. Una de las más utilizadas es **PySR** [5], una biblioteca de código abierto que combina un motor evolutivo en Julia con una interfaz en Python, ofreciendo operadores matemáticos protegidos —versiones de funciones como la raíz cuadrada o la división que evitan valores indefinidos (NaN o infinitos) durante la búsqueda— para garantizar la estabilidad numérica.

A continuación se formaliza el problema de regresión simbólica desde un punto de vista matemático.

Dado un conjunto de datos  $\mathcal{D} = \{(\mathbf{u}_i, v_i)\}_{i=1}^N$  donde:

- $\mathbf{u}_i \in \mathbb{R}^d$  es el vector de variables de entrada del  $i$ -ésimo punto del conjunto de datos,
- $v_i \in \mathbb{R}$  es el valor observado de la variable de salida para dicho punto,

se busca una función  $f : \mathbb{R}^d \rightarrow \mathbb{R}$  dentro de un espacio de funciones  $\mathcal{F}$  que minimice una función de pérdida  $\mathcal{L}$ , la cual mide la discrepancia entre las predicciones del modelo y los valores observados:

$$f^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f; \mathcal{D}). \quad (6)$$

El espacio  $\mathcal{F}$  está compuesto por todas las funciones que pueden construirse combinando un conjunto de operadores matemáticos básicos (suma, resta, multiplicación, división, logaritmos, etc.) y constantes numéricas. La tarea de encontrar la función óptima es intrínsecamente combinatoria: el número de expresiones que pueden formarse crece de manera exponencial con la cantidad de operadores y variables disponibles, y se ha demostrado que el problema de regresión simbólica es NP-duro [14]. Por esta razón se requieren métodos de búsqueda heurística, como los algoritmos evolutivos.

La implementación más extendida de estos algoritmos para regresión simbólica es la programación genética, cuyos fundamentos se exponen a continuación.

### 1.2.2. Fundamentos de programación genética

Una de las estrategias más utilizadas para la regresión simbólica es la programación genética, una rama de los algoritmos evolutivos inspirada en la selección natural [9]. Existen también enfoques basados en recocido simulado [8], gramáticas estocásticas [13] y redes neuronales [4, 6], aunque la programación genética sigue siendo una de las opciones más extendidas por su flexibilidad para explorar espacios de expresiones algebraicas [9, 5].

En este paradigma, cada posible expresión matemática se codifica como un árbol sintáctico: una estructura jerárquica en la que los nodos internos contienen operadores matemáticos (suma, multiplicación, seno, etc.) y las hojas contienen variables de entrada o constantes numéricas. La figura resultante tiene forma de árbol invertido, donde la raíz es el operador principal y las ramas conducen a los operandos.

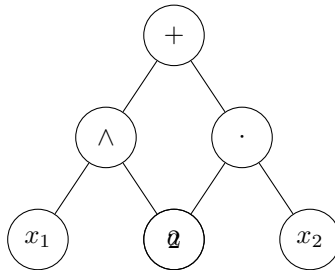


Figura 1: Árbol sintáctico de la expresión  $x_1^2 + a \cdot x_2$ .

La programación genética trabaja con una población de individuos (árboles) que se modifica a lo largo de generaciones —ciclos iterativos del proceso evolutivo— mediante los siguientes operadores:

- **Selección:** Los individuos que producen predicciones más cercanas a los valores reales tienen mayor probabilidad de pasar a la siguiente generación.
- **Cruce:** Dos árboles padres intercambian subárboles para generar descendientes que combinan características de ambos.
- **Mutación:** Se reemplaza aleatoriamente un nodo o subárbol por otro nuevo, introduciendo diversidad en la población.
- **Elitismo:** Una fracción de los mejores individuos de cada generación se preserva sin cambios en la siguiente, evitando así que se pierdan las soluciones más prometedoras. Este mecanismo es opcional y su intensidad depende del problema [9].

La repetición de estos operadores a lo largo de las generaciones permite explorar el espacio de posibles expresiones algebraicas, guiando la búsqueda hacia funciones que se ajustan cada vez mejor a los datos [9].

La calidad de cada individuo se mide mediante una función de pérdida, cuyas variantes más relevantes para este trabajo se presentan a continuación.

### 1.2.3. Funciones de pérdida

La **función de pérdida** —también llamada función de aptitud en el contexto de los algoritmos evolutivos— es el criterio que se utiliza para evaluar cuantitativamente qué tan buena es una expresión candidata producida por la regresión simbólica. Esta función mide la discrepancia entre las predicciones de la expresión simbólica y los valores observados en el conjunto de datos. Una función de pérdida común, y la que se emplea como referencia en este trabajo, es el error cuadrático medio.

**Error cuadrático medio.** Para un conjunto de datos  $\{(\mathbf{u}_i, v_i)\}_{i=1}^N$  con  $\mathbf{u}_i \in \mathbb{R}^d$  y  $v_i \in \mathbb{R}$ , y una función  $f : \mathbb{R}^d \rightarrow \mathbb{R}$ , el error cuadrático medio (MSE, por sus siglas en inglés) se define como

$$\text{MSE}(f) = \frac{1}{N} \sum_{i=1}^N (v_i - f(\mathbf{u}_i))^2, \quad (7)$$

donde:

- $N$  es el número total de puntos en el conjunto de datos,
- $v_i$  es el valor observado para el  $i$ -ésimo punto,
- $f(\mathbf{u}_i)$  es el valor predicho por la expresión  $f$  para el vector de entrada  $\mathbf{u}_i$ .

El MSE penaliza cuadráticamente las desviaciones, por lo que es sensible a valores atípicos [7]. En el contexto de la regresión simbólica, es una de las funciones de pérdida más utilizadas debido a su simplicidad y a que proporciona una señal suave para guiar el proceso de búsqueda [1].

La efectividad del proceso evolutivo no solo depende de la función de pérdida elegida, sino también de la configuración del algoritmo. A continuación se describen los parámetros que controlan el comportamiento de la programación genética aplicada a la regresión simbólica.

### 1.2.4. Parámetros del algoritmo de regresión simbólica

El comportamiento de la regresión simbólica basada en programación genética depende de un conjunto de parámetros que influyen en la calidad de las soluciones, el tiempo de cómputo y la interpretabilidad de los resultados [5]. A



continuación se describen los más relevantes, agrupados según el aspecto del algoritmo que controlan.

#### Parámetros de la población.

- **Tamaño de la población ( $P$ ):** Número de individuos que coexisten en cada generación. Una población grande aumenta la diversidad genética y la probabilidad de encontrar soluciones de alta calidad, a cambio de un mayor tiempo de cómputo y consumo de memoria.
- **Número de generaciones ( $G$ ):** Cantidad máxima de iteraciones. Un número elevado permite explorar más a fondo el espacio de búsqueda, aunque también incrementa el costo computacional.
- **Número de poblaciones independientes:** En algunas implementaciones se utiliza un modelo de islas, que consiste en dividir la población total en varias subpoblaciones que evolucionan en paralelo. Periódicamente, un pequeño número de individuos migra entre las islas, lo que ayuda a mantener la diversidad genética y evita la convergencia prematura [16].

**Parámetros de control de la complejidad.** Además de la precisión, interesa obtener expresiones sencillas que sean fáciles de interpretar. Los siguientes parámetros regulan el equilibrio entre ajuste a los datos y simplicidad de las fórmulas.

- **Coeficiente de parsimonia ( $\alpha$ ):** Penalización añadida a la función de pérdida, proporcional al tamaño del árbol (número de nodos). La pérdida efectiva se calcula como

$$\mathcal{L}_{\text{eff}}(f) = \text{MSE}(f) + \alpha \cdot \text{tamaño}(f).$$

Un valor mayor de  $\alpha$  favorece expresiones más simples.

- **Tamaño máximo del árbol ( $S_{\text{máx}}$ ):** Límite en el número de nodos que puede tener un individuo. Evita la explosión de la complejidad y reduce el espacio de búsqueda.

**Probabilidades de operadores genéticos.** La composición de cada nueva generación está determinada por las siguientes probabilidades, que deciden qué operador se aplica a cada individuo.

- **Probabilidad de cruce ( $p_c$ ):** Fracción de la nueva generación generada mediante cruce de padres seleccionados.
- **Probabilidad de mutación ( $p_m$ ):** Fracción de individuos creados mediante modificaciones aleatorias de los árboles existentes.

- **Probabilidad de reproducción** ( $p_r$ ): Fracción de individuos que pasan directamente a la siguiente generación sin cambios (elitismo), con  $p_r = 1 - p_c - p_m$ .

El proceso completo funciona de la siguiente manera: se genera una población inicial de árboles aleatorios, cada árbol se evalúa con la función de pérdida y se seleccionan los mejores para la siguiente generación. Mediante cruce y mutación se crean nuevos individuos, mientras que el elitismo preserva las mejores soluciones. Este ciclo se repite durante un número prefijado de generaciones o hasta que se alcanza una calidad suficiente, dando como resultado una expresión analítica que aproxima los datos.

Para analizar estadísticamente los resultados obtenidos con este proceso, en el capítulo de experimentación se emplea la prueba  $t$  pareada, cuyos fundamentos se presentan a continuación.

### 1.2.5. Prueba $t$ pareada

En el Capítulo ?? se comparan distintas funciones de pérdida evaluadas sobre los mismos casos de prueba. Para determinar si una de ellas ofrece un desempeño significativamente mejor que otra se utiliza la *prueba  $t$  de Student para muestras pareadas* [10]. Esta prueba contrasta la hipótesis nula frente a la hipótesis alternativa, que se formulan como

$$H_0 : \mu_d = 0 \quad \text{frente a} \quad H_1 : \mu_d > 0 \quad \text{o} \quad H_1 : \mu_d < 0,$$

donde  $\mu_d$  es la media de las diferencias entre los dos métodos. La dirección de la desigualdad en  $H_1$  depende de si se quiere probar que un método supera al otro (en tasa de éxito) o que es más rápido (en tiempo de ejecución).

Si se denota por  $d_i = x_i - y_i$  la diferencia entre los resultados de dos métodos para el  $i$ -ésimo caso, el estadístico de prueba se calcula como

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad s_d = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})^2},$$

donde  $n$  es el número de pares.

Bajo el supuesto de que las diferencias  $d_i$  son independientes y provienen de una población aproximadamente normal —condición que puede verificarse mediante una prueba de normalidad como la de Shapiro-Wilk—, el estadístico  $t$  sigue una distribución  $t$  de Student con  $n - 1$  grados de libertad [10]. Esta distribución es simétrica y con colas más pesadas que la distribución normal estándar, especialmente para tamaños muestrales pequeños. El parámetro que determina su forma es el número de grados de libertad,  $\nu = n - 1$ , que refleja la cantidad de datos disponibles para estimar la variabilidad de las diferencias. Conforme  $\nu$  crece, la distribución  $t$  se aproxima a la normal.

A partir del valor observado de  $t$  y de los grados de libertad se obtiene el  $p$ -valor, que indica la probabilidad de obtener un resultado al menos tan extremo como el observado si la hipótesis nula fuera cierta. Fijado un nivel de significación  $\alpha = 0.05$ , la decisión se toma comparando el estadístico  $t$  con el valor crítico  $t_{\alpha,\nu}$  que deja una probabilidad  $\alpha$  en la cola correspondiente de la distribución  $t$  de Student. Si la hipótesis alternativa es  $H_1 : \mu_d > 0$ , se rechaza  $H_0$  cuando  $t > t_{\alpha,\nu}$ ; si la alternativa es  $H_1 : \mu_d < 0$ , se rechaza  $H_0$  cuando  $t < -t_{\alpha,\nu}$ . Equivalentemente, se rechaza  $H_0$  si el  $p$ -valor es menor que  $\alpha$ .

En los análisis del Capítulo ?? se aplica la prueba  $t$  pareada para comparar tanto la tasa de éxito como el tiempo de ejecución de las distintas funciones de pérdida, indicando en cada caso el valor del estadístico  $t$  y el correspondiente  $p$ -valor.

Con los fundamentos presentados en este capítulo —sistemas no lineales, métodos numéricos, regresión simbólica y prueba  $t$  pareada— se cuenta con las herramientas necesarias para desarrollar la metodología que se presenta en el capítulo siguiente.

## Referencias

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [2] Charles G. Broyden. A class of methods for solving nonlinear simultaneous equations. *Mathematics of Computation*, 19(92):577–593, 1965.
- [3] Richard L. Burden and J. Douglas Faires. *Numerical Analysis*. Brooks/Cole, 9th edition, 2010.
- [4] Allan Costa, Régis Pacheco, Ralph Deters, and Luis C. Lamb. Occamnet: A fast neural model for symbolic regression at scale. *arXiv preprint arXiv:2007.10799*, 2021.
- [5] Miles Cranmer. PySR: Fast & parallelized symbolic regression in Python/Julia. *Journal of Open Source Software*, 8(89):5308, 2023.
- [6] Yusong Deng, Min Wu, Lina Yu, Jingyi Liu, Shu Wei, Yanjie Li, and Weijun Li. Operator feature neural network for symbolic regression. *arXiv preprint arXiv:2408.07719*, 2024.
- [7] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition, 2009.
- [8] Daniel Kantor, Fernando J. Von Zuben, and Fabricio Olivetti de Franca. Simulated annealing for symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '21)*, 2021.
- [9] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, USA, 1992.
- [10] Douglas C. Montgomery and George C. Runger. *Applied Statistics and Probability for Engineers*. Wiley, 7 edition, 2017.
- [11] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [12] Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009.
- [13] Léo François Dal Piccol Sotto and Vinícius Veloso de Melo. A probabilistic linear genetic programming with stochastic context-free grammar for solving symbolic regression problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '17)*, 2017.
- [14] Marco Virgolin, Tanja Alderliesten, and Peter A. N. Bosman. Symbolic regression is NP-hard. *arXiv preprint arXiv:2207.01018*, 2022.

- [15] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.
- [16] Darrell Whitley, Soraya Rana, and Robert B. Heckendorn. The island model genetic algorithm: On separability, population size and convergence. *Journal of Computing and Information Technology*, 1990.